SSE Proactive Assignment

Threat Modeling for Cloud-Based Online Payment System

Group 3

- SUNITHA KANUMURI (ID: 2024tm93188)
- KONAKANCHI NAVYA SREE (ID: 2024tm93184)
- A KAVYA (ID: 2024tm93117)
- MANAPURAM BHARGAVI (ID: 2024tm93081)
- HARMALKAR RAHUL RAJAN SAYALI (ID: 2024tm93073)

Step 1: Industry and Technology Platform Definition

Industry Focus: Financial Services (Digital Payment Processing)

The modern evolution of financial services, marked by widespread digital adoption, mandates the development of resilient and secure online payment systems. This report focuses on applying threat modeling to a core **Online Payment Platform** designed to facilitate customer fund transfers, bill payments, and secure e-commerce transactions across web and mobile channels. The handling of sensitive financial data makes proactive security analysis critical for maintaining regulatory compliance and customer confidence.

Technology Platform and Reference Architecture

The platform employs a highly available **Microservices Architecture** hosted entirely on the **AWS Cloud**, optimized for real-time transaction processing and horizontal scalability.

Component	Technology / AWS Service	Function / Purpose
User Interfaces	React (Web) & Native Mobile Apps	The customer-facing layer responsible for transaction initiation and interaction.
Business Logic	Java Spring Boot REST APIs	Core services executing transactional logic (e.g., Payment Processor Microservice).

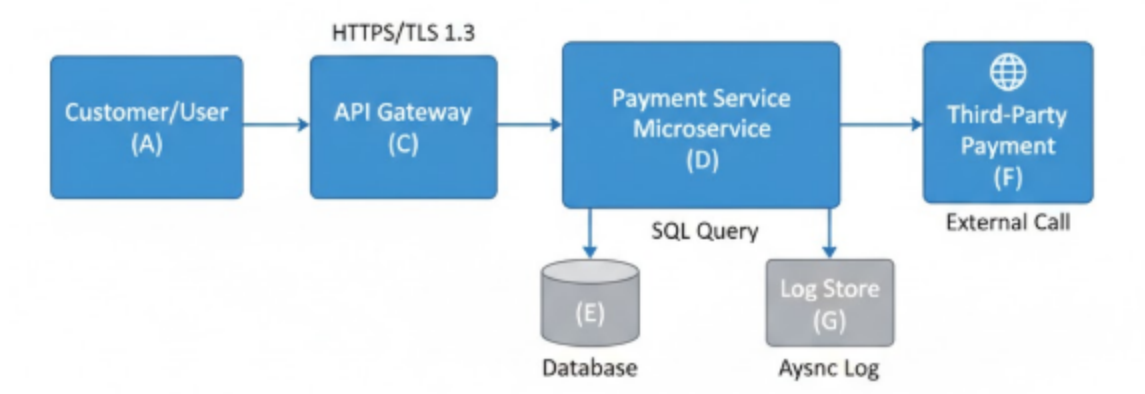
Data Storage	PostgreSQL (AWS RDS) & MongoDB	PostgreSQL for high-integrity, ACID-compliant transactional data; MongoDB for application and security logs that require high write throughput.
Cloud Services	AWS (API Gateway, EC2, Lambda, S3, KMS)	Manages external API ingress, resource hosting, serverless compute, and cryptographic key management.
DevOps Pipeline	Jenkins & GitHub Actions	Orchestrates automated integration, testing, and continuous deployment activities.
Security Foundation	AWS IAM, KMS, WAF	Provides fundamental identity and access control, key management, and network perimeter defenses.

Step 2: Threat Modeling and Security Requirements

The threat modeling process utilizes a structured methodology: **Define, Diagram, Identify, Mitigate, and Validate**. This stage centers on the **Diagram** and the **Identify/Mitigate** phases.

Artifact 1: Data Flow Diagram (DFD) and Trust Boundaries

The following DFD graphically illustrates the path of sensitive transaction data and delineates the critical interaction points and trust boundaries of the system.



Artifact 2: Threat Identification (STRIDE Analysis) & Mitigation

The STRIDE analysis identifies systematic threats targeting the system's components and data flows. To provide technical rigor, each threat is cross-referenced with relevant CWE and **OWASP Top 10** categories.

Category	Component/Data Flow	Example Threat	Mitigation Strategy (Control)	CWE / OWASP Mapping
S – Spoofing	Customer Login, Session Tokens	An attacker gains unauthorized access by compromising credentials or session identifiers.	Implement **Multi-Factor Authentication (MFA)** and enforce cryptographically signed, short-lived tokens (JWTs) for all sessions.	CWE-287 / Broken Auth (A07)

T – Tampering	Transaction Requests (D $\leftrightarrow$ F)	A malicious actor modifies transaction values or recipient details while data is in transit to external networks.	Utilize **HMAC or Digital Signatures** (PKI) on all outbound transaction payloads to guarantee integrity and authenticity.	CWE-353 / Insecure Output Handling
R – Repudiation	Customer Transactions	A customer fraudulently denies initiating a successful fund transfer.	Implement comprehensive, non-repudiable **Audit Logging** (in MongoDB) that records transaction status, source IP, unique user ID, and system confirmation (T $\rightarrow$ G).	CWE-778 / Repudiation Risk
I – Information Disclosure	Database (E), Backups	Sensitive PII and financial data are leaked due to unencrypted storage or a vulnerability like SQL Injection.	Mandate **Encryption at Rest** (AWS KMS managed RDS encryption) and utilize **Field-Level Encryption** for maximum protection of critical fields.	CWE-359 / Exposure of Private Info
D – Denial of Service	API Gateway (C)	A high-volume automated attack exhausts the application's	Deploy an **AWS WAF** with Geo-blocking and L7 filtering, coupled with	CWE-400 / Uncontrolled Resource Consumption

		resources, preventing service availability for legitimate users.	API Gateway's inherent throttling mechanisms.	
E – Elevation of Privilege	Payment Microservice (D)	A service defect allows a lower-privileged user to execute a higher-privileged action, such as administrative endpoint access.	Enforce strict **Role-Based Access Control (RBAC)** across all internal microservice APIs and adhere to the **Principle of Least Privilege** for application IAM roles.	CWE-269 / Improper Privilege Management

Proposed Security-Specific Requirements

The following non-functional requirements are proposed to directly mitigate the identified threats:

Requirement ID	Requirement Type	Description	Mitigation Link (STRIDE)
SEC-001	Authentication	The system MUST enforce Multi-Factor Authentication (MFA) for all login attempts and transactions exceeding a configurable threshold (e.g., $5,000).	Spoofing (S)
SEC-002	Authorization	All internal microservice endpoints MUST authenticate client	Elevation of Privilege (E)

		service identity and validate authorization scopes prior to execution.	
SEC-003	Data Integrity	Cryptographic signing MUST be applied to all transaction payloads sent to third-party processors.	Tampering (T)
SEC-004	Data Protection	All sensitive Personally Identifiable Information (PII) and financial data MUST be protected by AES-256 encryption at rest, managed by AWS KMS.	Information Disclosure (I)
SEC-005	Auditing	The application MUST generate immutable, time-stamped audit records for all security-relevant events (logins, logouts, transactions) (C $\rightarrow$ G, D $\rightarrow$ G).	Repudiation (R)
SEC-006	Availability	The Network Edge MUST enforce concurrent request rate limiting (e.g., 10 RPS per source IP) to safeguard against volumetric attacks.	Denial of Service (D)

Step 3: Recommendations for Development and Operations

Security must be treated as a continuous, integrated element of the Secure Software Development Life Cycle (SSDLC). The following recommendations guide the teams toward establishing a proactive security posture.

A. Secure Development (Dev) Recommendations

1. **DevSecOps Automation (SAST/DAST):** Implement **Static Application Security Testing (SAST)** and **Dynamic Application Security Testing (DAST)** tools directly into the CI/CD pipeline (Jenkins/GitHub Actions). The CI/CD process must implement build gate failures to block deployment if critical security defects, especially those related to **Injection (OWASP A03)** or **Insecure Design (OWASP A04)**, are identified.
2. **Strict Input Validation Enforcement:** Developers must treat all external data as untrusted. Enforce stringent server-side input validation and canonicalization on *all* API parameters to mitigate cross-site scripting (XSS) and SQL injection vulnerabilities.
3. **Mandatory Secret Management:** Eliminate hardcoding of credentials. Development policy must dictate that **all secrets** (database passwords, API keys, certificates) are provisioned and retrieved at runtime exclusively from **AWS Secrets Manager**.
4. **Dependency Vulnerability Scanning:** Utilize tools like OWASP Dependency-Check during the build process to scan and prohibit the use of open-source libraries containing known CVEs, mitigating supply chain risk.

B. Secure Operations (Ops) Recommendations

1. **Enforcement of Least Privilege (IAM):** Regularly audit AWS IAM policies to enforce the **Principle of Least Privilege**. Operational roles for microservices must be narrowly scoped, granting only the essential permissions required for their specific function (e.g., specific rds:connect and kms:decrypt actions).
2. **Network Microsegmentation:** Utilize AWS Security Groups (SGs) and Network ACLs (NACLs) to implement granular network microsegmentation. Communication between any two services must be explicitly whitelisted, strictly adhering to a zero-trust network model.

`

3. **Centralized Monitoring and SIEM:** Centralize all logs (API Gateway, WAF, application, and OS logs) into a Security Information and Event Management (SIEM) solution. Implement automated, high-priority alerting for security events such as brute-force attempts, database read anomalies, and suspicious IAM activity.
4. **Configuration and Patch Management:** Automate the patching and configuration management of the underlying operating systems (EC2 hosts for Java Spring Boot) using services like AWS Systems Manager (SSM). This ensures that critical CVEs are addressed promptly and system configurations remain hardened against baseline standards.

Conclusion

The conclusion of this proactive threat modeling exercise confirms its fundamental role in securing the Online Payment Platform's design and ongoing assurance. By linking STRIDE threats to tangible security requirements (SEC-001 through SEC-006) and integrating robust DevSecOps practices, the system achieves a state where security is an inherent, continuous quality, promoting a resilient operational environment essential for the regulated financial sector.